Ethics, Law & Responsible Data Collection

Week 2 · Foundations module · Textbook Chapter 2

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

August 24, 26 & 28, 2026

This week at a glance

Before we collect a single row, we settle the harder question — not *can* we, but **should** we, and if so, how responsibly?

Monday | Concepts & notebook intro

- The legal landscape, `robots.txt`, and a five-question ethical framework

Wednesday | Notebook lab

- `robots.txt`, User-Agent headers, rate limiting — and building `responsible_get()`

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 2 — our first real PR

Companion notebook

`ch-02-ethics.ipynb`

Bring a laptop

Wednesday is hands-on — we send live requests and watch what the server sees.

1. Monday · Concepts

The ethics of retrieval

"Web scraping" is a popular phrase — and a faintly pejorative one. Data is **valuable**: other people invested time, money, and effort to create and host what you are about to retrieve programmatically.

So the question is never simply **"can I scrape this?"** but **"should I — and if so, how do I do it responsibly?"**

This chapter builds a **decision framework** to revisit before every collection project in this course and beyond. Three layers of guidance, none sufficient alone: **law**, **technical norms** (`robots.txt`), and **ethics**.

One idea to carry all semester

"It's technically possible" is not the same as "it's legal," and "it's legal" is not the same as "it's ethical."

Textbook Ch. 2, "The Ethics of Retrieval."

The legal landscape is unsettled

The **Computer Fraud and Abuse Act** (CFAA, 1986) bans "unauthorized access" — but what that means for scraping is still litigated. Three cases draw today's boundaries:

- **Van Buren v. United States (2021)** — the Supreme Court's "gates-up-or-down" reading: the CFAA targets entering areas you may not enter, *not* misusing data you were allowed to see.
- **hiQ Labs v. LinkedIn (2019, 2022)** — scraping *public* profiles likely isn't a CFAA violation (no login gate bypassed), yet hiQ still **lost on breach of contract** for scraping after agreeing to the ToS.
- **Sandvig v. Barr (2020)** — violating a site's Terms of Service is not, by itself, a federal crime — modest cover for audit-study research.

The CFAA has also been used to *threaten* researchers and journalists — the Aaron Swartz case is the sobering example.

Textbook Ch. 2, "Legal Frameworks." Legal-ethical analysis of ToS-based restrictions: Fiesler et al. (2020).

A moving target: scraping, AI & privacy law

As of mid-2026, three developments matter most for your work:

- **Public data, contract risk.** Meta's claims against the broker **Bright Data** (2024) failed — the scraping collected only public, logged-out data — and a parallel **X Corp.** suit was dismissed. Legal risk concentrates at *authentication barriers and contracts*, not public visibility.
- **The AI front.** *New York Times v. OpenAI* and a wave of similar suits ask whether scraping to **train models** is fair use. Publishers have already hardened sites and added AI directives to `robots.txt`.
- **State privacy law.** California's CCPA/CPRA and 12+ states — including the **Colorado Privacy Act** — now bring GDPR-like rights. If your dataset names residents, these laws can apply to *you*, even as a student.

Practical implication

Treat personal data as a **liability, not an asset** — collect the minimum you need.

Textbook Ch. 2, "The Current Landscape." Data revolts against AI scraping: (Frenkel et al. 2023).

An ethical decision framework

Before any project, work through five questions:

1. **Legality** — CFAA, GDPR, local law, the site's ToS?
2. **Consent** — did people consent? Is it truly public, or shared with a reasonable expectation of limited visibility (Zimmer 2010)?
3. **Proportionality** — only the data you need? Could a smaller or less sensitive set answer the question?
4. **Privacy** — any PII? Could individuals be re-identified? What if it leaks?
5. **Server impact** — a meaningful load? Rate-limited? Scraping off-peak?

No framework substitutes for judgment — but working these questions surfaces the risks.

IRBs & web data

Scraping becomes *human-subjects research* when you collect data about **identifiable people** to produce **generalizable knowledge**. Public officials' votes: usually not. Social-media profiles for health or politics: almost certainly. When in doubt, ask your IRB.

Textbook Ch. 2, "An Ethical Decision Framework" & "IRBs and Web Data." The Belmont Report maps consent → respect, proportionality/privacy → beneficence, risk → justice.

2. Wednesday · Notebook Lab

`robots.txt`: read it before you scrape

A `robots.txt` at a site's root declares what automated agents may fetch. Read it raw, or parse it and ask permission per URL:

```
import requests
from urllib.robotparser import RobotFileParser

print(requests.get("https://en.wikipedia.org/robots.txt").text[:400])
# User-agent: * / Disallow: /w/ / Allow: ... / Crawl-delay: ...

rp = RobotFileParser()
rp.set_url("https://en.wikipedia.org/robots.txt")
rp.read()

rp.can_fetch("*", "https://en.wikipedia.org/wiki/Python_(programming_language)") # True
rp.can_fetch("*", "https://en.wikipedia.org/w/api.php") # False
rp.crawl_delay("*") # seconds between requests, or None -> pick a polite default
```

The nuance: `/wiki/` articles are open, but `/w/` (dynamic pages + API) is disallowed *for crawlers*. Deliberate API clients follow different norms (Ch. 10) — `robots.txt` is one input, not the whole rulebook.

Textbook Ch. 2, "Technical Norms: `robots.txt`."

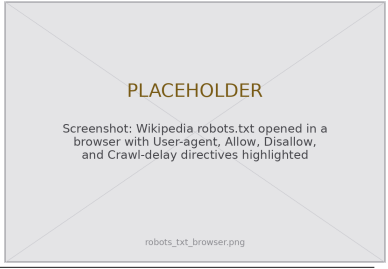
What `robots.txt` reveals across platforms

```
sites = {"Wikipedia": "https://en.wikipedia.org/robots.txt",
         "Reddit": "https://www.reddit.com/robots.txt",
         "NY Times": "https://www.nytimes.com/robots.txt"}

for name, url in sites.items():
    lines = requests.get(url).text.strip().split("\n")
    disallow = sum(l.startswith("Disallow") for l in lines)
    allow = sum(l.startswith("Allow") for l in lines)
    print(f"{name}: {len(lines)} lines, {disallow} Disallow, {allow} Allow")
```

Wikipedia — permissive where it matters (openness is the mission; data flows through its API + dumps). **Reddit —** tolerates access within limits. **NY Times —** restrictive; the content is a commercial product. Reading it tells you *what kind of organization* you're dealing with.

Two caveats: it's a **norm, not a law** (violating it isn't illegal, but it's rude and gets you IP-blocked); and a **404** means *no rules*, not "crawl aggressively."



Textbook Ch. 2, "Comparing `robots.txt` Across Platforms."

Identify yourself, then slow down

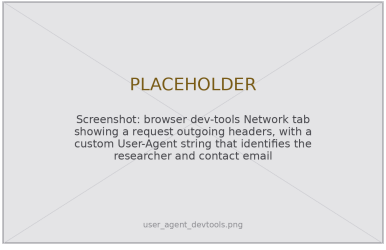
Two habits make you a good citizen of the web — a truthful `User-Agent` and rate limiting:

```python
import time, requests

headers = {
    "User-Agent": "WebDataScience/1.0 (INFO 4617; brian.keegan@colorado.edu)"
}

for url in urls:
    response = requests.get(url, headers=headers)
    # ... process the response ...
    time.sleep(1) # a good default: about one request per second
```

The default `requests` User-Agent marks you as an anonymous bot. A good one **identifies you and gives contact info**, so operators can reach you instead of simply blocking you — and some APIs *require* it. Hundreds of requests per second is an accidental **denial-of-service**: respect any `Crawl-delay` and stay well inside documented API limits.



Your header is visible to every server you touch.

Textbook Ch. 2, "Responsible Request Headers" & "Rate Limiting." HTTP mechanics: *Missing Manual* Ch. 24.

`responsible_get()`: build it once

Custom User-Agent, rate limiting, and error handling appear in *every* scraping project. Wrap them into one reusable function instead of re-implementing them ad hoc:

```python
import requests, time

def responsible_get(url, headers=None, delay=1.0):
    """GET with responsible defaults: identify, rate-limit, fail loudly."""
    default_headers = {
        "User-Agent": "WebDataScience/1.0 (INFO 4617; you@colorado.edu)"}
    if headers:
        default_headers.update(headers)
    try:
        response = requests.get(url, headers=default_headers, timeout=30)
        response.raise_for_status() # treat 4xx/5xx as failures, not data
        time.sleep(delay) # rate-limit by default
        return response
    except requests.exceptions.RequestException as e:
        print(f"Request failed ({type(e).__name__}): {url}")
        return None
```

Textbook Ch. 2, “Building a Responsible Scraper Function.”

Why it's built that way — and reused everywhere

Every default encodes a principle

- **Custom User-Agent** — identify & give contact
- `delay` **defaults ON** — aggressive scraping must be a *conscious* override, not an oversight
- `raise_for_status()` — a 403/404 fails loudly instead of feeding your parser garbage
- broad `except` — `ConnectionError`, `Timeout`, `SSLError` log rather than crash the run
- `timeout=30` — never hang forever on a dead server
- pass a site's `Crawl-delay` as `delay` to honor its stated preference

You'll import this again

`responsible_get()` returns in **Ch. 6** (scraping HTML tables), **Ch. 7** (Wayback snapshots), and throughout the **API chapters**.

Building responsible defaults into your *tools* is more reliable than remembering to add them each time.

Textbook Ch. 2. Reused in later chapters on static pages, archives, and APIs.

Lab: work these, then show-and-tell

Work through the notebook exercises in pairs:

1. Compare `robots.txt` for five sites in one category — what patterns emerge?
2. ToS audit — find three clauses restricting automated collection; do research exceptions apply?
3. Run the five-question framework on a scenario: scraping a dating app's profiles to study self-presentation
4. Rate-limit experiment — time five requests with `time.time()` at 1s then 2s; does timing match intent?
5. Audit a site you use daily: reconcile `robots.txt`, ToS, and the framework in 300 words — where do they conflict?
6. 5617: read Fiesler et al. (2020) + EFF's CFAA explainer + *Van Buren*; write a two-page platform-collection memo

Show-and-Tell

Come ready to share:

- a challenging bug
- interesting data you pulled
- a provocation for a research design

“Can I scrape this?” is the wrong question.

Ask instead: *should* I — and if so, how do I do it responsibly?

Legality · Consent · Proportionality · Privacy · Server impact.

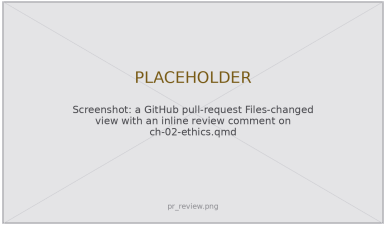
3. Friday · Textbook Revisions

Improving Chapter 2 together

Last week you set up Git and GitHub. Today we open our **first real pull request** — against `ch-02-ethics.qmd` — and review a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable



Contributions & reviews count toward the Textbook Revisions grade (30%).

A revision menu for Chapter 2

High-value targets — pick one and scope it to a single PR:

- **Re-run the live code.** `robots.txt` files drift — verify the described Wikipedia `/w/` rules and Reddit patterns still hold, then update the prose and expected output.
- **Close a gap in "Common Issues to Debug."** Add a worked **403-with-a-good-User-Agent** case, a `robots.txt` 404 example, or a note on CAPTCHA / bot-detection beyond `robots.txt`.
- **Add a figure.** A clean five-question framework diagram, an annotated `robots.txt` anatomy, or a CFAA case timeline.
- **Strengthen an exercise.** Add a rubric or expected output to the framework exercise, or a starter cell so it self-checks.
- **Update the legal landscape.** The "Current Landscape (mid-2026)" section dates quickly — verify a recent ruling or add a Colorado Privacy Act explainer tied to the "Public Interest Connection" callout.

Targets drawn from the chapter's callouts, "Common Issues," and "Further Reading."

What makes a good revision & a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **The Post-API Age** — why the open web is closing, and how researchers adapt.

Key takeaways

1. **"Can I?" is not "should I?"** — the real question is how to collect data *responsibly*.
2. The law is unsettled: the CFAA targets **bypassing gates** (Van Buren), but **contracts and ToS** still bite (hiQ); state privacy laws and GDPR reach personal data.
3. `robots.txt` is a **norm, not a law** — read it, respect `Disallow` and `Crawl-delay`; a 404 means no rules, not open season.
4. **Identify yourself** (custom `User-Agent`) and **rate-limit** (`time.sleep`) — then wrap both into `responsible_get()` and reuse it.
5. Run the **five-question framework** before every project; when in doubt, use an API, collect less, and err toward caution.

Reading: Textbook Ch. 2. Related: Fiesler et al. (2020); Freelon (2018).